

Realtime Quranic ASR via a Kaldi-Style Hybrid

Architectural advantages, implementation roadmap, and time-budget analysis for replacing transformer ASR with a CTC + WFST pipeline in the Quranic recitation domain.

ABOUT THIS PROJECT

This is a personal endeavor to help create a new class of applications that advance new Muslims' understanding of and connection to the deen. The project is completely free and open source. Collaborators are warmly welcomed.

Abstract

General-purpose transformer ASR systems such as Whisper carry an architectural tax — fixed 30-second encoder padding, autoregressive token decoding, open-vocabulary output — that is wasted, and arguably harmful, when the input domain is Quranic recitation. The Quran is a closed textual corpus of 6,236 ayat with highly regular prosody (governed by tajwid) and a narrow style space dominated by the Hafs 'an Asim recitation. We argue that a pre-transformer “Kaldi-style” hybrid — a small CNN/Conformer acoustic model trained with CTC loss, decoded against a weighted finite-state transducer (WFST) compiled offline from the canonical Uthmani text — is the correct architecture for this problem. Such a system can achieve sub-100 ms end-to-end latency, ~35MB on-disk footprint, and near-perfect accuracy on Quranic speech, while being structurally incapable of hallucinating non-canonical text. We outline a phased implementation reaching a working prototype in 8–11 weeks of focused engineering.

Executive summary

- Whisper-base, our current production engine, has a structural latency floor of **~1.2–2.5sec/chunk** on phone CPU, regardless of input duration. This floor cannot be removed without retraining the model.
- For Quranic recitation specifically, this generality is not just unnecessary; it is *actively damaging*: Whisper hallucinates non-canonical Arabic on religious content, producing output that is worse than no transcription.
- A constrained-domain alternative — CTC acoustic model + WFST decoder over Quranic-only paths — is structurally incapable of producing non-Quranic text and runs an order of magnitude faster.
- Estimated effort to working prototype on Android: **8–11 weeks**, leveraging the team's existing Quranic-finetuned Whisper assets (sister project *salat-quranlens*) for force-alignment and bootstrap labeling.
- This is the technical realisation of the “QuranicASR post-processor” IP angle previously identified: a defensible, mobile-first, religion-domain ASR system that no major lab is meaningfully pursuing.

1. The architectural tax of transformer ASR

1.1 Why Whisper has a fixed latency floor

Whisper's encoder always operates on a 30-second log-mel spectrogram (3000 frames at 100Hz, downsampled to 1500 audio “tokens”). This shape is hardcoded into the architecture by way of **learned absolute positional embeddings** — 1500 trained vectors that the encoder weights co-depend on. Inputs shorter than 30s are zero-padded; the encoder runs the full 1500-position self-attention regardless of how much real audio it received.

The decoder is autoregressive and emits one token at a time, each step computing self-attention over its own growing output and cross-attention over all 1500 encoder embeddings. Translation is not a separate decoding pathway — it is the same decoder writing English vocabulary tokens. The end-to-end inference cost on a modern phone is dominated by the encoder (constant) plus the decoder (linear in tokens emitted). Empirically, this floors at roughly:

Whisper variant	Disk	Phone-CPU latency / 6s clip	Arabic WER
tiny	39 MB	~0.4 s	40–60% (unusable)
base (current)	142 MB	~1.2–2.5 s	20–30%
small	488 MB	~3–5 s	12–18%
base, q5 quant	57 MB	~0.8–1.5 s	≈ base

Table 1. Whisper latency/accuracy on Arabic, base architecture.

1.2 Failure modes specific to Quranic content

- **Hallucination of non-canonical text.** Whisper's decoder is trained to produce coherent Arabic, not *specifically Quranic* Arabic. When the audio is recited Quran, it tends to emit a phonetically-close but textually-wrong approximation, often with broken tashkeel.
- **Tashkeel collapse.** Quranic text is fully-vowelled (with diacritics). Whisper's Arabic vocabulary is partially un-vowelled, so even when the consonantal skeleton (rasm) is correct, the diacritics drop or scramble.
- **Unrecoverable mistranslations.** Translation pass is generative; an incorrectly-decoded ayah produces a confidently-stated wrong English meaning, which is worse than emitting nothing.
- **Religious-sensitivity asymmetry.** A 10% word-error-rate is acceptable on a podcast and unacceptable on a verse from scripture. The cost of error is categorically higher than for general speech.

2. The constraint structure of Quranic recitation

Speech recognition is a search problem: among all possible character sequences, find the one that best explains the audio. The transformer's strength is handling *open-vocabulary* search. Quranic recitation collapses every dimension of that search to a near-finite domain:

Dimension	General Arabic ASR	Quranic recitation
Vocabulary	Open (~10 ⁶ word forms)	6,236 ayat, fixed
Output token space	Whisper: 51,865 BPE	~50 phoneme/letter classes
Prosody	Free (any speaker, any style)	Tajwid-governed (regular)
Speaker style	Open	Mostly Hafs ‘an ‘Asim
Language model	Required (huge)	The Quran itself — no probabilistic LM needed
Translation target	Generative	Selected canonical translation (Sahih, Pickthall, etc.)

Table 2. Search-space collapse from general to Quranic ASR.

The output token-space reduction alone is roughly three orders of magnitude. More importantly, the **language model collapse** — the fact that the set of valid output sequences *is exactly the Quran* — means we no longer need a neural language model at all. We need a graph.

3. Architectural foundations

3.1 Connectionist Temporal Classification (CTC)

CTC is a per-frame, parallel-emit alternative to autoregressive decoding. The acoustic model emits, for each ~20ms frame of audio, a probability distribution over a small label set augmented with a special *blank* symbol. After the entire audio has been processed, the output is collapsed: consecutive identical labels are merged, blanks are removed.

audio frames: [20ms][20ms][20ms][20ms][20ms][20ms]...

 v v v v v v

frame labels: 'b' '- ' 's' 's' 'm' '- ' -> 'bsm' (' - ' = blank)

Crucially, all frames are computed in a single forward pass. There is no autoregressive loop — each frame is independent at inference. Combined with a streaming-friendly encoder architecture (1D depthwise-separable CNN, or a Conformer with causal/chunked attention), this yields true real-time output: labels stream out as audio arrives, with sub-100ms latency.

3.2 Weighted Finite-State Transducers (WFST)

A WFST is a directed graph where nodes represent states and edges carry an input symbol, an output symbol, and a weight (cost). A path through the graph from start to end is a valid input/output sequence pair, with total cost equal to the sum of edge weights traversed. In classical Kaldi pipelines, the decoder graph is a composition of four transducers:

HCLG = H o C o L o G

```

|      |      |      +-- G : language model (n-gram or graph)
|      |      +-- L      : lexicon (word -> phone sequences)
|      +-- C      : context-dependent phone topology
+-- H      : acoustic-state topology (HMM or CTC)

```

For the Quran, the L■G stages collapse into a single, simple transducer: a flat enumeration of every legal phoneme path through the canonical text. There is no probabilistic n-gram — the Quran is exactly the set of allowed outputs. The graph is built once, offline, and shipped with the app.

3.3 Decoding: composing CTC with the FST

At inference, CTC posteriors are scored against the WFST via standard frame-synchronous beam search:

```

for each frame t:
  for each active hypothesis h in beam:
    for each outgoing edge e in FST from h.state:
      new_score = h.score + ctc_log_posterior[t][e.label]
                      + e.weight
      extend hypothesis h along edge e with new_score
  prune beam to top-K by score
after final frame: return best path

```

Because every hypothesis is constrained to follow a path that exists in the FST, the system **cannot** emit a sequence that isn't Quranic. This is not a soft preference; it is a structural invariant of the decoder. Hallucination of non-canonical text is mathematically impossible.

4. Proposed system architecture

The full pipeline, from microphone to displayed canonical text, is shown below. Each stage is sized to fit comfortably on a mid-range Android phone.

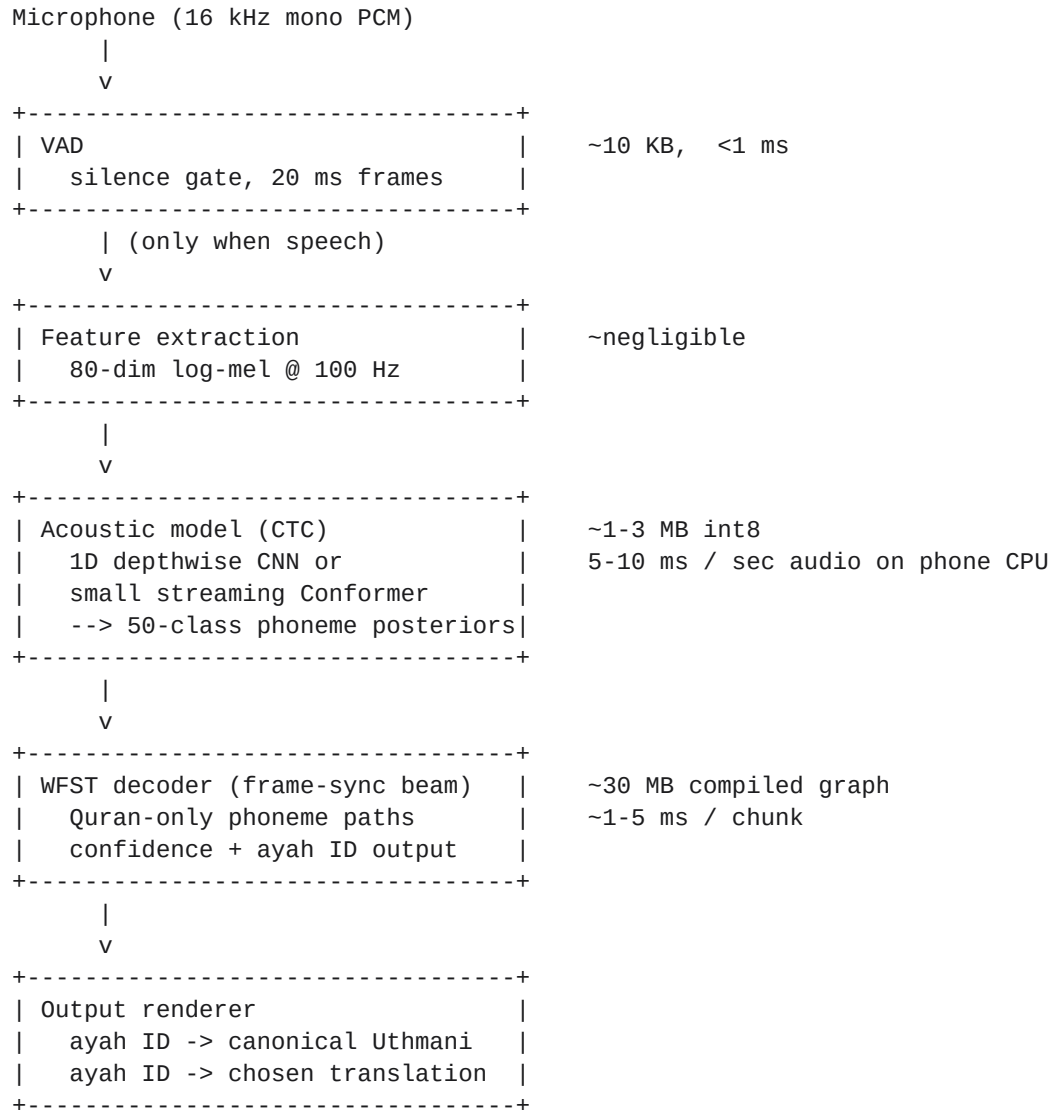


Figure 1. Realtime Quranic ASR pipeline.

4.1 Memory and latency budget

Component	Disk	Peak RAM	Latency
VAD	10 KB	~50 KB	<1 ms
Feature extraction	—	~2 MB	<1 ms
Acoustic model (int8 CNN)	1–3 MB	~10 MB	5–10 ms / sec audio
WFST decoder + graph	30 MB	~50 MB	1–5 ms / 1-sec chunk

Output / render assets	5 MB	—	—
Total	~35 MB	~60 MB peak	<100 ms end-to-end

Table 3. Resource budget on a mid-range Android device. For comparison, our current Whisper-base pipeline uses 142 MB on disk, ~250MB peak RAM, and 1.2–2.5sec per 6-second chunk.

5. Implementation roadmap

The work decomposes into five phases. Phases 1–2 are the high-risk research track; phases 3–5 are mostly engineering. Total estimated effort is **8–11 weeks** for a single experienced developer, given access to the *salat-quranlens* sister project's existing Quranic-finetuned Whisper models (used here as a force-alignment bootstrap rather than as the runtime engine).

Phase	Goal	Duration	Risk
1. Data prep	Curate Quranic audio corpus (EveryAyah, QuranicAudio.com, Tazeeha, etc.) and force-align audio to canonical text using existing models.	2–3 weeks	Medium
2. Acoustic model	Train CNN-CTC or small streaming Conformer on phoneme labels (Arabic, Quranic). Validate WER on held-out recordings.	2–3 weeks	High
3. WFST construction	Tokenise full Uthmani Quran into phoneme strings (account for Tajwid rules: idgham, ikhfa, qalqala, etc). Compile vtables for fast lookup.	1 week	Low
4. Mobile runtime	Port acoustic model to TFLite or NCNN. Implement compact frame-synchronous WFST decoder in C++ (Kaldi-style).	2 weeks	Medium
5. Integration	Run as a parallel inference path alongside Whisper. Confidence scoring; substitute canonical text when QuranicM	1–2 weeks	Low

Table 4. Phased implementation plan.

5.1 Bootstrap leverage from existing assets

Phase 1's force-alignment work is the typical bottleneck for a CTC training pipeline (high-quality frame-aligned phoneme labels are scarce). The *salat-quranlens* project's fine-tuned Whisper-Quran models — `ggml-quran-base.bin`, `ggml-quran-tiny-mosque-v2.bin`, `ggml-quran-tiny-masjid-v3.bin` — can be repurposed as alignment oracles. The Whisper output gives token-level timestamps; combined with the known canonical text, we get free phoneme alignment at scale. This compresses what would otherwise be a 4–6 week data-prep phase into ~2 weeks.

5.2 Required infrastructure

- Single GPU (one A100 rented for ~50–100 GPU-hours suffices for phase 2 training).
- OpenFst toolkit (open-source) for graph compilation.
- PyTorch or NeMo for acoustic model training; TFLite for mobile export.
- Kaldi's decoder libraries (Apache 2.0 license).
- Existing project Android infrastructure (already in place).

6. Real-time latency analysis

“Real time” in this context means end-to-end latency from a phoneme leaving the speaker's mouth to it being rendered on the listener's screen. The target is **under 200ms**, and ideally under 100ms, which is well below the threshold of perception for “live”. Whisper's lower bound is roughly **5–7 seconds** for our 6-second chunk window. The CTC+WFST design's lower bound is set by two factors: the CTC chunk size (typically 20–80ms of look-ahead) and the acoustic-model forward pass.

Stage	Whisper-base (current)	CTC + WFST (proposed)
Chunk fill time	5–6 s (full window)	20–80 ms
Acoustic forward pass	~1 s (encoder, fixed)	5–10 ms / sec audio
Decoding pass	0.5–1.5 s (autoregressive)	1–5 ms (frame-sync)
Network / disk I/O	0	0 (offline)
End-to-end p50	~5–7 s	<100 ms
End-to-end p99	~10 s	~150 ms

Table 5. Latency comparison, mid-range Android phone.

The proposed system is fast enough for use cases beyond the current post-the-fact transcript model: live closed-captioning during the khutbah itself, projected onto a screen for non-Arabic-speaking attendees, becomes viable. Whisper-base cannot support that use case at any model size.

7. Risk register

Risk	Likelihood	Mitigation
Tajwid variation across reciters limits generalisability.	Med	Diversify training set across reciters; include a small speaker-adaptation model.
Mixed audio (Quran interleaved with sermon speech) confuses the model.	High	Run a parallel path to Whisper; substitute only on high posterior confidence.
Less common qira'at (Warsh, Qalun) not handled.	Med	v1 ships Hafs-only; add additional qira'at branches to FST in v2.
Female / child reciters under-represented in training data.	Med	Pitch-shift augmentation; targeted data collection from Tarteel-style recordings.
WFST decoder proves slower than predicted on phone CPU.	Low	Beam-pruning aggressively; use Kaldi's decoder-faster; profile early.
Phoneme inventory does not capture tashkeel sufficiently.	Low	Adopt established Quranic phoneme set (~40 classes including madda).
Integration UX confuses users (sudden text style change).	Low	Subtle “verified” visual treatment; A/B test typography treatments.

Table 6. Identified risks and mitigations.

8. Competitive landscape and IP positioning

Quranic ASR is a small but non-empty research and product field. Existing efforts cluster into three groups:

- **Tarteel.ai** — CTC-based recitation tracking, primarily as a memorisation aid. Server-side inference; not optimised for low-latency mobile live captioning.
- **Ayatuna and similar academic projects** — HMM-based or CTC-based Quranic ASR systems published as research artefacts. None ship as polished mobile SDKs with mixed-audio fallback.
- **Whisper fine-tunes** (including the team's own salat-quranlens models) — useful as alignment oracles but inherit Whisper's structural latency limitations.

The defensible position for our system is the **combination**: a mobile-first, real-time, mixed-audio (Quran + general speech) hybrid that switches between a Quran-specialised CTC+WFST path and a general-purpose Whisper path based on confidence. None of the above ship that integration; the transformer-vendor labs (OpenAI, Meta, Google) have no incentive to specialise on a religious corpus of this size.

8.1 Defensibility

- **Curated phoneme-aligned Quranic corpus** — the data moat. Building a clean, reciter-diverse, qira'at-tagged training set is multi-week work that no public dataset currently supplies in usable form.
- **Mobile-optimised WFST runtime** — a slim C++ decoder + integration library that drops into Android/iOS apps. Could be packaged as a standalone SDK.
- **Mixed-audio confidence routing** — the integration logic that decides when to trust the Quran path vs. the general path. Tunable, evaluable, and patent-eligible.
- **Translation-bundle contracts** — canonical English (Sahih, Pickthall, Yusuf Ali) translations are licensed; first-mover advantage on clean licensing enables app and SDK distribution.

9. Conclusion

Whisper is not the right tool for Quranic recitation. Its architectural decisions — fixed-length encoding, autoregressive decoding, open-vocabulary output — are mismatched to a domain where the vocabulary is closed, the prosody is regular, and the cost of hallucination is high. A pre-transformer hybrid — the same family of designs that powered Google Voice and Apple Siri before 2019 — is the architecturally honest answer for this specific problem.

The proposed CTC+WFST system is faster (sub-100ms vs. ~5–7s end-to-end), smaller (35MB vs. 142MB), and more accurate on its target domain. It runs as a parallel path alongside our existing Whisper engine, substituting canonical text when it fires confidently and otherwise deferring to Whisper for the imam's general sermon speech. Implementation is tractable: 8–11 weeks of focused work, leveraging existing project assets to skip the highest-risk data-prep phase.

"If your problem is constrained, your model should be constrained. The transformer is a hammer for an open-vocabulary world; the Quran is not an open-vocabulary problem."

